

PROJECT REPORT

EVENT MANAGEMENT SYSTEM

Submitted By	Akshat Shukla
Registration Number	25BAI10309
School	SCAI
University	VIT Bhopal University
Project Type	Java Desktop Application

A project report prepared according to the prescribed project documentation guidelines.

Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. References
15. Evaluation Rubric Alignment

Note: The supplied project screenshot is included in the Screenshots / Results section. The report describes the visible Java desktop interface and its event/participant management workflow without assuming an external database that is not evidenced in the supplied material.

1. Introduction

The **Event Management System** is a Java-based desktop application designed to simplify the basic activities involved in managing events and their participants. The system provides a graphical user interface (GUI) through which a user can enter event information, register participants, display records, and remove records when required.

The project focuses on applying core programming and object-oriented design concepts to a practical problem. Instead of maintaining event information manually, the user interacts with a single application window containing input fields, action buttons, and a result/display area.

The supplied interface contains fields for **Event ID, Event Name, Event Date, Venue, Participant ID, Participant Name, and Participant Email**. It also provides operations for adding, viewing, and deleting events, as well as registering, viewing, and deleting participants.

The main objective is to create a simple, understandable system that demonstrates event record management and participant registration while keeping the interface accessible to a beginner-level desktop application user.

2. Problem Statement

Managing events manually can become inconvenient when the number of events and participants increases. Information may be written in different places, repeated unnecessarily, or become difficult to locate quickly. A small computerized system can centralize these operations and provide a consistent way to add, view, and delete records.

Problem definition: Develop a desktop-based Event Management System that allows a user to maintain event details and participant details through a graphical interface and perform common record-management operations.

The project addresses the following practical issues:

- Entering event information in a structured format.
- Keeping participant information associated with event-management activities.
- Providing quick access to all currently available event records.
- Registering and displaying participants without requiring separate manual lists.
- Removing outdated or incorrect event and participant records.
- Providing a clear interface so that operations can be performed without command-line interaction.

Project scope

The current scope is intentionally focused on core CRUD-style operations for events and participants. Advanced capabilities such as online payments, email notifications, authentication, cloud deployment, and multi-user synchronization are outside the current scope and are listed as possible future enhancements.

3. Functional Requirements

The functional requirements describe what the application should allow the user to do.

ID	Requirement	Description
FR-01	Add Event	Accept event ID, name, date and venue, then create an event record.
FR-02	View All Events	Display the event records currently managed by the application.
FR-03	Delete Event	Remove a selected event record using its identifying information.
FR-04	Register Participant	Accept participant ID, name and email and register the participant.
FR-05	View Participants	Display participant information available to the application.
FR-06	Delete Participant	Remove an existing participant record.
FR-07	Clear	Clear the input fields so a new operation can be entered.
FR-08	Exit	Close the application safely.

4. Non-functional Requirements

The non-functional requirements define the expected quality and usability characteristics of the system.

Usability: The interface should be simple enough for a first-time user to understand. Labels are placed beside input fields and operations are represented by clearly named buttons.

Performance: For the intended small-scale use, event and participant operations should respond quickly because the application performs lightweight record-management operations.

Reliability: The application should handle normal user operations consistently and avoid unintended changes to unrelated records.

Maintainability: The project should use separate logical responsibilities for the GUI and event/participant management so that individual features can be modified without rewriting the complete interface.

Portability: As a Java desktop application, the system can be run on a compatible Java environment rather than being tied to a single web browser.

Readability: Field names and buttons use descriptive terms, making the purpose of each control apparent.

5. System Architecture

The application can be represented using a simple layered architecture. The graphical interface receives input from the user, application logic performs the requested event or participant operation, and the data-handling layer maintains the records used by the application.

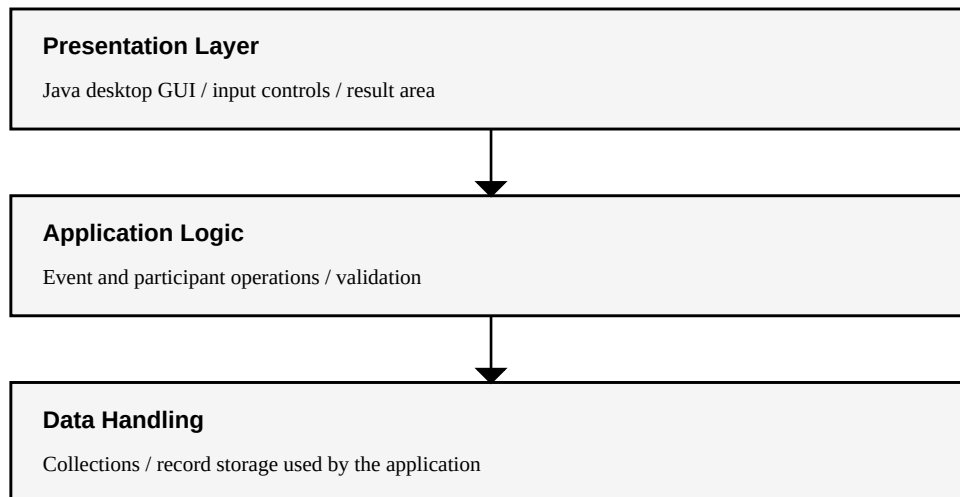


Figure 1. High-level architecture of the Event Management System.

Main architectural components

- **Presentation layer:** Java desktop GUI containing labels, text fields, buttons and a large output area.
- **Application logic:** Controls the operations for adding, viewing, deleting and registering records.
- **Data handling:** Maintains the event and participant information during application use. The supplied material does not establish a requirement for an external database, so this report does not claim one.

The architecture is deliberately small because the project is intended to demonstrate a complete working desktop application without introducing unnecessary infrastructure.

6. Design Diagrams

The following diagrams describe the major interactions and structural elements of the proposed system.

6.1 Use Case Diagram

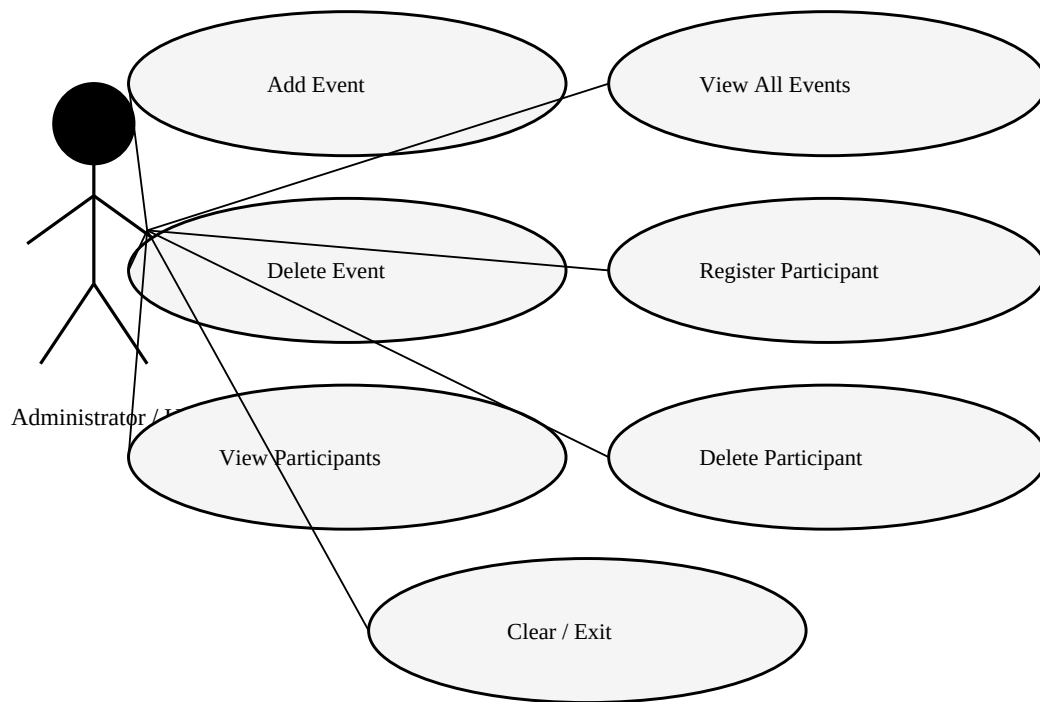


Figure 2. Use case view showing the main operations available to the user.

6.2 Workflow Diagram

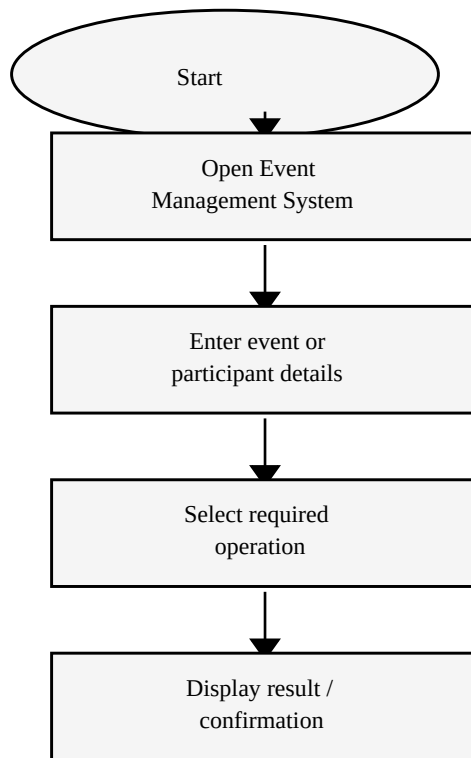


Figure 3. Basic workflow from entering information to displaying the result.

6.3 Sequence Diagram

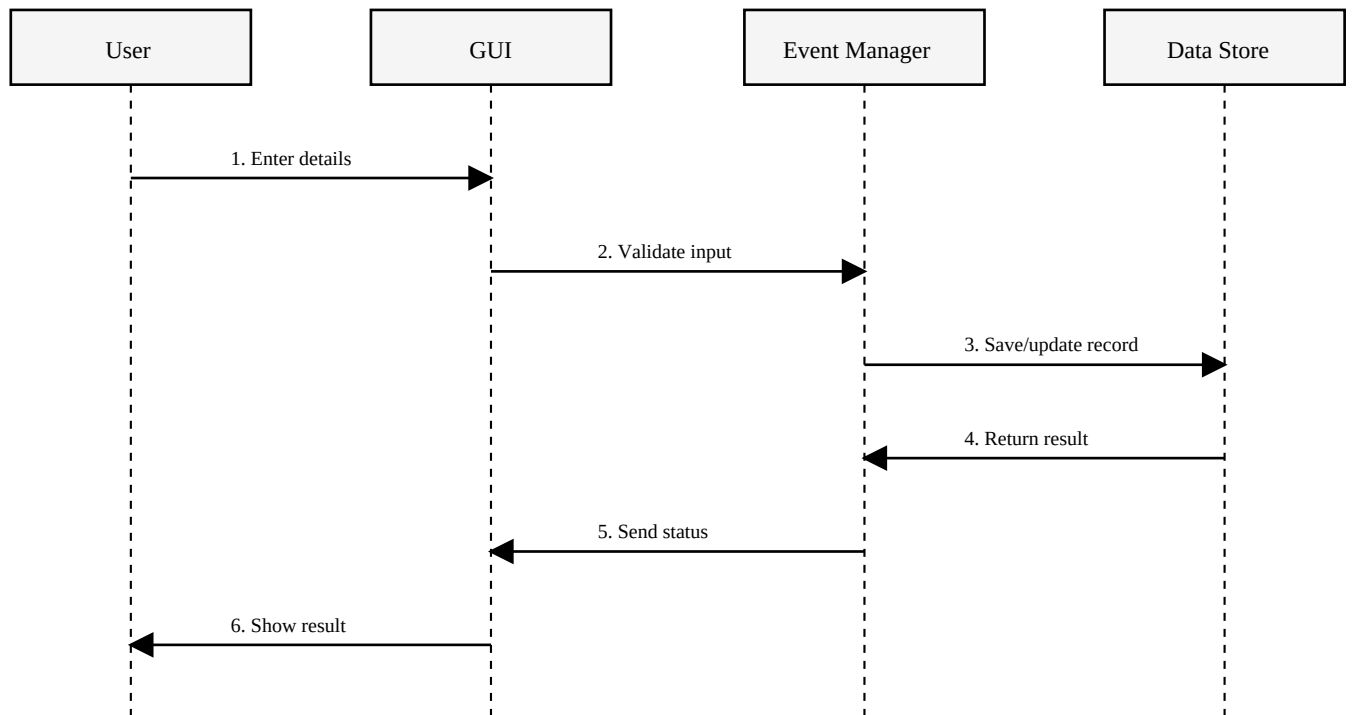


Figure 4. Sequence of interaction between the user, GUI, application logic and data handling.

6.4 Class / Component Diagram

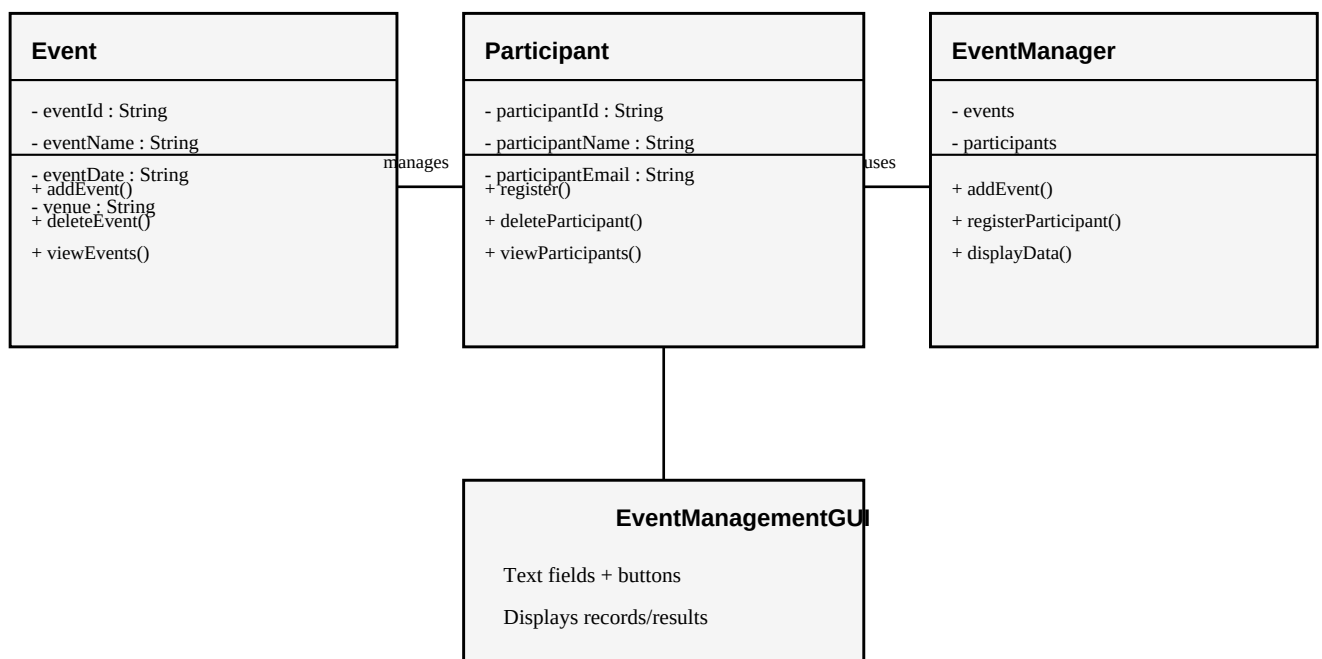


Figure 5. Simplified class/component view of the application.

6.5 ER Diagram

An ER diagram is not included as a database-specific design because the supplied project evidence does not establish a persistent relational database. If a database is added later, the natural entities would be **Event** and **Participant**, with a registration relationship connecting participants to events.

7. Design Decisions & Rationale

Desktop GUI: A desktop interface was selected because it makes the project visually demonstrable and allows the user to perform all major operations from one window.

Single-window layout: The visible interface keeps event and participant inputs together, reducing the need to navigate between multiple screens.

Separate event and participant operations: Events and participants are different logical records, so their operations are represented by separate buttons.

Text-field based input: Text fields are straightforward to implement and allow the user to enter IDs, names, dates, venue details and email addresses directly.

Dedicated output area: A large display area is provided for showing records and operation results without opening additional windows for every action.

Clear and Exit controls: These controls improve usability by allowing the user to reset the form or close the program explicitly.

The overall design favors clarity over excessive complexity. For an academic project, this makes the individual programming concepts easier to demonstrate during evaluation and easier to maintain.

8. Implementation Details

The supplied GUI is a Java desktop application interface. The structure is consistent with a Java Swing-style application: the window contains labels, text fields, buttons and a multiline display area. The implementation can be organized around event-management and participant-management responsibilities.

Input fields

- Event ID — identifies an event record.
- Event Name — stores the name/title of the event.
- Event Date — stores the scheduled date.
- Venue — stores the location of the event.
- Participant ID — identifies a participant.
- Participant Name — stores the participant's name.
- Participant Email — stores the participant's email address.

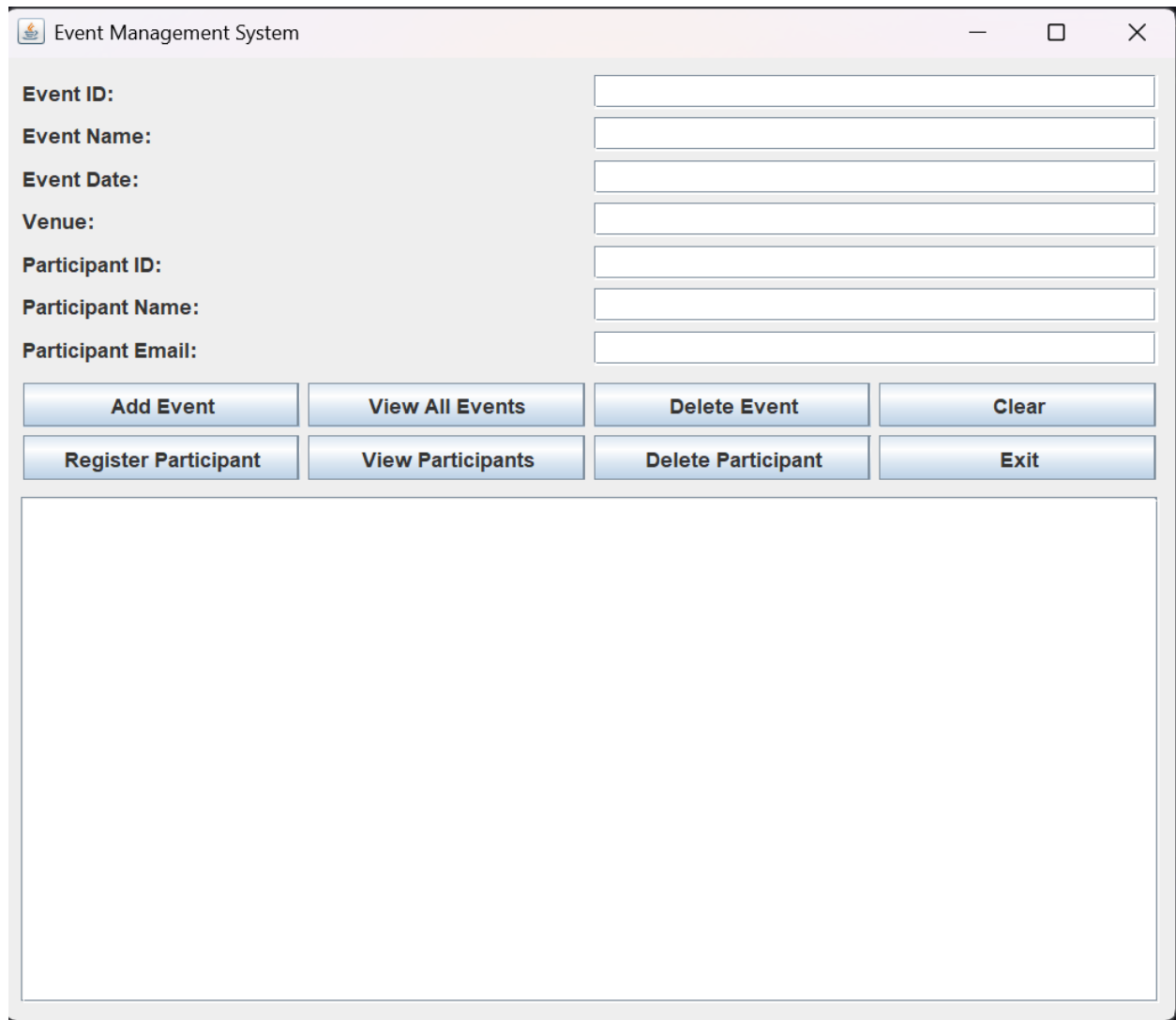
Operations

Operation	Expected processing
Add Event	Read event fields, validate required input, create/store the event and report the result.
View All Events	Read currently managed event records and display them in the output area.
Delete Event	Use event identification information to find and remove the relevant record.
Register Participant	Read participant details, validate required input and store the participant.
View Participants	Display the participant records maintained by the application.
Delete Participant	Find and remove the relevant participant record.
Clear	Reset the text fields.
Exit	Terminate the application window.

Validation consideration: In a complete implementation, empty IDs and required fields should be rejected before records are created or deleted. Email input can also be checked for a basic valid format. These checks improve reliability without changing the user interface.

9. Screenshots / Results

The following screenshot is taken from the supplied project interface and demonstrates the implemented graphical front end of the Event Management System.



The screenshot displays a window titled "Event Management System". The interface is organized into several sections:

- Input Fields:** Seven labeled text boxes for data entry:
 - Event ID:
 - Event Name:
 - Event Date:
 - Venue:
 - Participant ID:
 - Participant Name:
 - Participant Email:
- Action Buttons:** Two rows of four buttons each:
 - Row 1: Add Event, View All Events, Delete Event, Clear
 - Row 2: Register Participant, View Participants, Delete Participant, Exit
- Display Area:** A large, empty rectangular box at the bottom, intended for showing records or operation output.

Figure 6. Supplied screenshot of the Event Management System GUI.

Observed interface elements

- Window title: Event Management System.
- Seven labeled input fields for event and participant information.
- Eight clearly named action buttons covering event operations, participant operations, clearing and exit.
- A large lower display area intended for showing records or operation output.
- The layout places data-entry controls above the action area, giving the application a simple form-based workflow.

10. Testing Approach

Testing was planned around the user-visible functions of the system. Each button should be tested with normal input, missing input, repeated operations and invalid identification values.

Test ID	Test case	Input / Action	Expected result
T01	Add valid event	Enter all event fields and click Add Event	Event is added and a result/status is displayed.
T02	View events	Click View All Events	Current event records are displayed.
T03	Delete event	Enter an existing event ID and click Delete Event	Matching event is removed.
T04	Register participant	Enter participant details and click Register Participant	Participant is registered.
T05	View participants	Click View Participants	Current participant records are displayed.
T06	Delete participant	Enter an existing participant ID and click Delete Participant	Matching participant is removed.
T07	Clear form	Enter sample values and click Clear	Input fields become empty.
T08	Exit	Click Exit	Application closes.
T09	Missing required data	Leave required field(s) empty and submit	Application should reject the operation and inform the user.
T10	Unknown ID	Delete using an ID that does not exist	No unrelated record is removed; user receives an appropriate message.

Testing should be repeated after any change to the record-handling logic or GUI event handlers to ensure that existing functionality is not broken.

11. Challenges Faced

Designing a usable GUI: Arranging several fields and buttons in one window while keeping labels readable and controls aligned requires careful layout decisions.

Managing two record types: The project handles both events and participants, so the program must keep their fields and operations logically separated.

Input validation: User-entered IDs, names, dates and email addresses may be empty or incorrectly formatted. Handling these cases prevents confusing results.

Delete operations: Deletion must identify the correct record and avoid accidentally removing a different record.

Displaying results: The application needs a simple way to communicate successful operations, unavailable records and current data to the user.

Keeping the implementation understandable: Because the project is an academic Java application, the code should remain simple enough to explain during a demonstration while still separating responsibilities clearly.

12. Learnings & Key Takeaways

The project provides practical experience in the following areas:

- Designing a desktop GUI using Java components and event-driven interaction.
- Connecting buttons and input fields to application operations.
- Applying basic object-oriented thinking by representing events and participants as separate logical entities.
- Understanding CRUD-style operations: create/add, read/view, and delete.
- Organizing a project into presentation, application-logic and data-handling responsibilities.
- Considering validation and error cases instead of testing only ideal inputs.
- Designing diagrams such as use case, workflow, sequence and class/component diagrams before or alongside implementation.
- Testing individual features systematically using defined test cases.
- Preparing a project repository and documentation so that another person can understand the purpose and operation of the application.

A key takeaway is that even a small desktop application benefits from structured requirements, a clear interface, separation of responsibilities and planned testing.

13. Future Enhancements

The current system provides a compact foundation. The following enhancements could extend it into a more complete event-management product:

- **Persistent database:** Store events and participants in MySQL, SQLite or another relational database so records remain available after the application closes.
- **Event-participant relationship:** Allow a participant to be explicitly registered for a particular event and maintain registration history.
- **Search and filtering:** Search events by ID, name, date or venue and participants by ID or name.
- **Update operation:** Add edit/update functions for existing event and participant records.
- **Input validation:** Provide stronger date validation, email validation and duplicate-ID checks.
- **Authentication:** Add administrator/user login with role-based permissions.
- **Reports:** Generate participant lists, event summaries and attendance reports.
- **Notifications:** Add email or application notifications for upcoming events.
- **Improved GUI:** Add tables, date pickers, menus and confirmation dialogs while retaining a simple workflow.
- **Deployment:** Package the Java application so it can be installed and launched easily on compatible systems.

14. References

Reference	Use in project
Oracle Java Documentation	Reference for Java language, classes, collections and desktop application concepts.
Java Swing / AWT Documentation	Reference for GUI components, windows, buttons, text fields and event-driven interfa
UML design concepts	Reference for use case, sequence and class/component diagrams used to document
Project source code and supplied GUI screenshots	Primary project-specific material used to describe the implemented interface and visib

Recommended documentation sources for the final repository README:

- Java language and API documentation from Oracle.
- Documentation for the GUI toolkit used in the implementation.
- Any libraries or frameworks actually included in the final GitHub repository.

15. Evaluation Rubric Alignment

The project has been documented with the supplied evaluation categories in mind. The table below maps the report and implementation activities to each category.

Component	Weightage	How the project addresses it
Problem Understanding & Requirements	15%	Problem statement, functional requirements and non-functional requirements define the scope of the project.
Design & Documentation	20%	Architecture, use case, workflow, sequence and class/component diagrams document the system design.
Implementation Quality	25%	The GUI provides event and participant operations with structured inputs, controls and a responsive interface.
Innovation, Depth & Complexity	15%	The project combines two related record types and provides a complete set of event/participant management features.
GitHub Repository & Version Control	10%	The final repository should contain source code, a README, screenshots, project structure and a clear commit history.
Project Report	20%	This report covers the prescribed sections, implementation explanation, testing, screenshots, and a conclusion.
Total	100%	

Final submission checklist: Before uploading, ensure that the GitHub repository contains the complete Java source code, the project can be run using the stated Java version, the README explains setup and usage, screenshots are included, and the report PDF is uploaded separately on the portal.

— End of Report —